

Architecture for Local Summarization Overlay Extension

Browser Extension: DOM Scanning & Communication Pattern

The extension should use **Manifest V3** with a service-worker background script and a content script injected into pages. The **content script** scans the webpage DOM for items (e.g. `<article>` elements, post containers, etc.) and extracts their text and any images. This can be done by querying known selectors or semantic tags and then stripping HTML to get clean text ¹ ². For each item found, the content script gathers:

- **Text Content:** Inner text of the article/card element, cleaned of extra whitespace or hidden text (as in the Lumos example function that strips HTML and collapses whitespace ³).
- **Image Data (if present):** If an `<img>` is found inside the item, grab its `src`. The extension can convert it to Base64 (e.g. by fetching it as a blob and using `FileReader`) or send the URL to the backend. (For simplicity in v1, a Base64 string avoids CORS issues with the backend fetching the image.)
- **Meta Info:** (Optional) An ID or index for the item to correlate responses.

Because content scripts in Manifest V3 are **subject to page CORS restrictions**, they should not call the backend directly if the server doesn't allow it ⁴ ⁵. Instead, use **message passing**: the content script sends the extracted data to the **background script** (using `chrome.runtime.sendMessage` or similar) ⁶ ⁷. The background script, which has higher privileges, then performs the HTTP request to the local server. This pattern isolates DOM scraping in the content script while delegating network and heavy processing to the background ¹. The extension's manifest should include the appropriate host permission (e.g. `http://localhost:5000/*`) to let the background script call the local API ⁸.

When the background receives the analysis result from the backend (summary, image tag, ad flag), it relays it back to the content script (e.g. via `chrome.tabs.sendMessage` or the response callback of `sendMessage`). The **content script** then injects an overlay **panel/label** into the page for each item. This overlay can be a small DOM element appended to the item or absolutely positioned over it. It will display the 1-2 line summary and the 2-3 word image tag (if any), and it will be styled to indicate ad vs. organic content. For example, the extension might add a green border or icon for normal content and a red highlight or "Ad" label for flagged ads. Using simple CSS (which can be included in the content script bundle or inserted dynamically) ensures the overlay is visible but not obtrusive. The content script should also observe DOM changes (using a **MutationObserver**) if the site loads content dynamically (like infinite scroll), so new items get processed in turn – though for v1 this can be as simple as rescanning periodically to avoid complexity. Overall, this architecture keeps the extension front-end **lightweight and focused**, handing off heavy AI work to the local backend.

Local Backend Server: Framework & Routing Design

Use a lightweight local web server to handle requests from the extension and interface with the Ollama LLM. This backend could be a **Node.js Express** app or a **Python FastAPI** app – either can easily run on localhost with minimal overhead. For a clean v1, a single endpoint (e.g. `POST /api/analyze`) can handle the incoming data. The extension's background script will POST a JSON payload like:

```
{ "text": "<extracted item text>", "image": "<base64 image data or URL>", "id": "<item-id>" }
```

The backend endpoint should parse the JSON, then call the Ollama API to get the summary, image tag, and ad flag. With Express, you'd use `app.use(express.json())` to parse JSON; with FastAPI, define a Pydantic model for the request. The server should **return** a JSON response containing the results, for example:

```
{ "summary": "...", "image_tag": "...", "is_ad": false, "id": "<item-id>" }
```

This simple REST interface keeps the frontend-backend contract clear. In more complex versions, you might split endpoints (e.g. `/summarize` and `/classifyAd`), but combining them is faster for v1. The **routing** is minimal: one route that accepts the content and returns analysis. Internally, structure the code modularly – e.g. a function `analyzeItem(text, imageData)` that orchestrates the Ollama prompt call – so you can swap out the model or add caching later without changing the API. The backend does not need to maintain state between requests (each request is independent analysis), which simplifies implementation.

For performance, keep the backend lean: avoid unnecessary middleware and use asynchronous calls if possible. With Node, you can use `fetch()` or Axios to call the Ollama HTTP API; with Python, `httpx` or `requests` (in FastAPI, you might do this in a threadpool if using sync I/O). If using Ollama's official libraries ⁹, that can simplify calling the model. Ensure the server listens **only on localhost** (e.g. bind to 127.0.0.1) so it isn't exposed externally. This local backend approach is a proven pattern: for example, an extension can run a Node server on `localhost:3001` to handle data and avoid direct calls from the browser, bypassing CORS and keeping complex logic out of content scripts ¹⁰. Using a local backend also means no cloud dependency and no need for message queues or brokers – the extension just calls a local REST API and waits for the JSON result.

Ollama LLM Integration: Prompt Structure & API Calls

Leverage **Ollama's local LLM API** to perform the summarization, tagging, and ad detection in one shot. Ollama runs models locally and exposes a REST API (by default at `http://localhost:11434/api`) ¹¹ ¹². You can run the Ollama server by executing `ollama serve` (with the `OLLAMA_ORIGINS` env var as noted below for CORS) and then sending requests to its `/generate` or `/chat` endpoints. In the backend, prepare a **prompt** that clearly instructs the model to output the required fields. For example, a single prompt (for a chat-based model) could be constructed as:

System prompt: "You are a local assistant that summarizes content and identifies ads."

User prompt: "Summarize the following text in 1–2 sentences. If an image description is provided, suggest a 2–3 word tag for the image. Determine if the text sounds like an advertisement. Respond in JSON with keys `summary`, `image_tag`, and `is_ad`."

Then include the text (and possibly an image placeholder or description). Because we want a structured result, it's best to take advantage of **Ollama's structured output** capability. You can pass a special `format` parameter with value `"json"` or a JSON Schema describing the expected structure ¹³ ¹⁴. For example, send `format: "json"` and instruct the model to output exactly `{"summary": "...", "image_tag": "...", "is_ad": ...}`. This helps ensure the model's output can be parsed reliably (the Ollama docs note that structured outputs let you enforce a JSON schema on responses for consistent replies ¹⁴). Setting the model's temperature low (e.g. 0 or 0.2) will also make it follow the format more deterministically ¹⁵.

Many open-source models (like Llama 2 variants) can handle summarization well. For image tagging, you'll need a **vision-capable model**. If Ollama is running a multimodal model (such as one supporting image inputs, like `gemma3` in their documentation), you can include the base64 image in the API call. The Ollama API accepts an `images` array of base64-encoded images for models that support vision ¹³. In that case, you'd attach the image data and likely mention in the prompt that an image is provided (some multimodal models implicitly consider the image input without needing a textual mention). For example, using the HTTP API, your backend could POST to `http://localhost:11434/api/generate` a JSON like:

```
{
  "model": "your-model-name",
  "prompt": "<constructed prompt with instructions + content>",
  "images": [<base64-image-string>],
  "format": {
    "type": "object",
    "properties": {
      "summary": { "type": "string" },
      "image_tag": { "type": "string" },
      "is_ad": { "type": "boolean" }
    },
    "required": ["summary", "image_tag", "is_ad"]
  }
}
```

The structured `format` schema above explicitly tells the model what JSON keys and types to return (using Ollama's JSON schema support) ¹⁶ ¹⁷. The backend sends the full text (you might truncate if the text is extremely long to keep inference fast) and the image. The **LLM will then produce a JSON response** string containing the summary, a short tag for the image, and a boolean/ad marker. The Ollama server will return this in its JSON response under a field like `"response"`. Your backend should parse that (e.g. `JSON.parse` if using Node) to ensure it's valid JSON.

Using one model call to handle all tasks is efficient for v1 – we avoid multiple passes or multi-agent orchestration (no LangChain graphs or external queues as requested). If the chosen model isn't great at a

particular sub-task (say ad detection), you can tweak the prompt or in future swap in a specialized model just for that. But initially, a single well-crafted prompt to a decent 13B-ish model should suffice for summarizing a paragraph and making a simple classification. Keep the **prompt sharp and concise** to optimize speed: unnecessary verbosity in instructions can slow down inference. Also consider using a smaller model if latency is a concern – since everything runs locally, a 7B model will be faster than a 30B model (at the cost of some accuracy). This modular approach means later on you can introduce caching (e.g. don't re-summarize the same text twice) or upgrade the model without changing the extension.

Data Model & API Schema

Design a clear data model for the request and response payloads between the extension and backend. Using a **structured schema** helps both sides (extension and server) remain in sync and makes the system extensible. Define, for instance, a TypeScript interface or JSON schema for an “ItemAnalysis”:

```
// Type definitions for clarity
interface ItemAnalysisRequest {
  id: string;      // an identifier for the item (could be index or DOM id)
  text: string;    // extracted text content of the item
  image?: string;  // base64 image data (or URL if using URL approach)
}
interface ItemAnalysisResponse {
  id: string;      // echoes the item id for correlation
  summary: string; // 1-2 line summary of the text
  image_tag?:
string; // 2-3 word description of image (if image provided; could be null/empty
if no image)
  is_ad: boolean;  // flag indicating if the item is likely an ad
}
```

The extension will send an `ItemAnalysisRequest` for each content item, and receive an `ItemAnalysisResponse`. Using consistent keys and types is important. On the backend, you can enforce this with a library (e.g. Pydantic in FastAPI or Joi in Node) or at least clearly document it. In the extension, having a matching TypeScript type for the response helps ensure you handle the data safely. In the example of the Tomodachi Pets extension, they even factored shared types into a common file for the extension and backend to use ¹⁸ – a practice you could adopt so that both components use the same definition of the data format.

The **API schema** can be kept simple in v1: JSON in, JSON out over HTTP. There's no need for complex GraphQL or extra metadata. Ensure the endpoint returns proper HTTP status codes (200 for success, maybe 400 for bad input). Also, the backend should be forgiving in what it accepts – for example, if `image` is missing or empty, it should still handle the request (the prompt would just omit the image-related instruction or send a flag to the model that no image is present). The `id` field, while optional, is useful if you plan to send multiple items in parallel or as a batch; it lets the extension know which result belongs to which DOM element. In a batched scenario, you could send an array of items and return an array of results, but that adds complexity in parsing and coordinating with the content script. The straightforward approach

is one request per item, which is easier to implement and plenty fast if items are, say, news headlines or post excerpts.

By clearly defining the data model, you make the system **modular**. For instance, if later you want to plug in a different model or add fields (maybe a sentiment score or category label), both the extension and backend can be updated in tandem. Keeping this contract stable and versioned (even just a note in code comments or docs) will prevent confusion as the project grows.

Security Considerations & Localhost Interaction

Because this system is local, security concerns are relatively low, but there are a few best practices to follow:

- **CORS and Origins:** If the extension's background script is calling the local backend via `fetch`, the requests will have an origin like `chrome-extension://<extension-id>`. The local server should allow this origin. For an Express.js backend, you can use the `cors` middleware to allow the Chrome extension's origin (you might specify the exact `chrome-extension://<id>` or use a wildcard if needed). In the case of calling Ollama's API directly, Ollama by default has strict CORS. The solution (as noted by the Ollama team) is to launch it with an environment variable to permit the extension origin. For example, running: `OLLAMA_ORIGINS=chrome-extension://* ollama serve` will **bypass CORS restrictions for extension requests**¹⁹. This allows the Chrome extension (which has origin `chrome-extension://...`) to talk to Ollama's localhost API without getting blocked. Essentially, configure your local services to **whitelist the extension** rather than allowing all origins, to mitigate any malicious webpage from accessing them. Since the extension background is the one making requests, and it can be given extra privileges, this is mostly a concern if the backend might inadvertently accept connections from regular webpages. Binding to `127.0.0.1` (not `0.0.0.0`) already ensures remote websites can't directly hit the API, but a rogue script in a webpage could try to `fetch('http://localhost:5000')`. By enabling CORS only for the extension origin, you ensure such requests are denied by the browser.
- **Manifest Permissions:** Include only the necessary permissions in the extension manifest. For example, you might use `"host_permissions": ["http://localhost:5000/*"]` to allow connecting to the local server⁸. Also use `"activeTab"` or content script matches judiciously – e.g. `"content_scripts": [{ "matches": ["<all_urls>"], ... }]` if you truly need to run on all pages, or narrower patterns if focusing on specific sites. Least privilege principle helps maintain user trust (the extension should only read content it needs to summarize).
- **Localhost Security:** Running a local backend means the user has to trust that it's safe. Keep the backend process secure by not executing untrusted code. In v1, the backend simply forwards text to an LLM – ensure you're not accidentally running eval on any input. Also, be cautious with the model's output if you ever plan to use it in any executed context (for now, just displaying text is fine).
- **Adversarial Content:** Since the model runs locally, prompt injection or malicious content in pages is less of a data leak risk (no external AI service is seeing it), but it could still confuse the model. It's worth constraining the model's behavior via the prompt (as done) to not execute any tools or produce code. Also, consider a **timeout** on model calls – if a certain piece of content is extremely

long or causing the model to hang, the backend should timeout and perhaps return a default message rather than hang the extension.

- **Resource Usage:** Running an LLM locally can be CPU/GPU intensive. Advise users that this extension will utilize local resources. It might be wise to allow the user to toggle the extension on/off (so it's not always summarizing every page). This can be as simple as a browser action button that enables the overlay only when clicked, to avoid constantly running the model in the background. For v1, the focus is on local operation, so no user data leaves the machine – which is a big privacy win.

In summary, favor a locked-down local setup: only the extension can talk to the local AI backend, and only the localhost API can invoke the model. This keeps the system self-contained and secure.

Development Workflow Tips (Hot Reload, Logging, Debugging)

During development, streamline your workflow so you can iterate quickly on the extension and backend:

- **Live-reloading the Extension:** Manually reloading the unpacked extension for every change is tedious. Use a tool or approach for hot-reloading. One option is the **Extensions Reloader** Chrome extension, which lets you refresh all unpacked extensions with one click or even automate it via a specific URL ²⁰. If you are using a bundler (like Webpack or Vite) for the extension code, there are plugins available (e.g. *webpack-chrome-extension-reloader* or a Vite plugin for MV3) that establish a websocket to auto-reload the extension when you rebuild ²¹. Setting this up in development can save a lot of time – you can write code and see the changes in Chrome immediately. Remember that content scripts might need a page refresh to reinject after a reload, so the tooling should handle that (or you press F5 on the page).
- **Hot-reloading the Backend:** For the local server, use a watchdog to auto-restart on code changes. If it's a Node Express server, run it with `nodemon` in development so any file save restarts the server. For Python FastAPI, use `uvicorn --reload` which does the same. This way you don't have to manually stop/start the server whenever you tweak the prompt format or logging.
- **Logging:** Liberally add logging on both the extension and backend. In the extension, `console.log` statements in the content script will appear in the webpage's console (devtools) and in the background script they appear in the extension's service worker console. Use these to confirm the extension is finding the right elements and that it's sending/receiving data correctly. On the backend, log incoming requests and key actions (like the prompt sent to the model, and whether the model responded in time). This will help pinpoint if something goes wrong in the pipeline. You can view extension logs by inspecting the extension's background service worker in Chrome's Extensions page and also by using DevTools on the content script context ²² ²³. For the backend, just run it in a terminal during dev to see the console logs.
- **Debugging:** Chrome DevTools is your friend. You can set breakpoints in the content script by using the Sources tab when inspecting the page. For the background script, go to `chrome://extensions`, click "Service Worker" under your extension to inspect it (it opens a console for the background script where you can also debug). If using TypeScript with a bundler, ensure you generate source maps so that debugging is easier. On the backend, you can use a debugger or

simply use logging and testing. Also consider building a small test harness for the backend – e.g. using curl or Postman to send sample text to your `/api/analyze` endpoint and verifying the output, independent of the extension. This can isolate issues in prompt or model output without the complexity of the extension.

- **Development vs Production Config:** It can be useful to have configuration flags. For instance, a dev mode could use a smaller model or a dummy fast response. You might also enable more verbose logging or debugging interfaces in dev. Keep the code modular so you can easily switch model names or enable/disable certain features with a flag. For example, have a setting for `MODEL_NAME` (so you can swap models without code changes), or a toggle to skip actually calling the model and return mock data (handy when you want to test the extension UI without waiting on the LLM).
- **Hot Module Replacement (HMR) for UI (if applicable):** If your overlay has complex UI (maybe later versions use a React component for the overlay), set up your build system to support HMR so that changes reflect without full reload. In v1, likely the overlay is just created via DOM APIs, so this is not a big concern.
- **Testing on Various Pages:** As you develop, test the extension on a variety of sites (news site, social media, etc.) to ensure your DOM selection logic is robust. You may find you need to adjust what elements you capture (for instance, excluding text from comment sections or nav bars that got picked up accidentally). These tweaks are easier to do early on.

Throughout development, maintain focus on simplicity and speed. Each component should do one job clearly: the extension grabs content and displays results, the backend calls the model. By logging and hot-reloading, you'll quickly identify any bottlenecks. For instance, if summarization is slow, you might try a smaller model or optimize the prompt length. Keep an eye on the Chrome console for any errors (CORS issues, permission errors, etc.) and address them as they come – e.g. if you see a CORS error, update the server's allowed origin or the extension manifest accordingly. Using Chrome's developer mode and devtools will surface these issues so you can fix them. As a tip, always **load the unpacked extension from the manifest's directory** (not a sub-folder) and ensure the backend is running before testing features that call it ²⁴ ²⁵ .

By following these practices, you'll have a smooth dev experience building v1. The result will be a clean, modular system: a Chrome extension that locally processes page content and overlays helpful summaries and tags in real-time, all without cloud services. This foundation will be easy to extend in future versions.

Sources:

1. Andrew Nguonly – “Local LLM in the Browser Powered by Ollama”, Medium (Nov 2023) – on using a Chrome extension with Ollama for local inference ¹ ¹⁹ .
2. *Ollama Documentation* – API usage for local models, including image input and JSON output formatting ¹³ ¹⁴ .
3. **MDN Web Docs** – Chrome extension content scripts vs. background script communication and permissions (Manifest V3) ⁴ ⁵ .
4. Tomodachi Pets Extension (GitHub) – Example of a Chrome MV3 extension communicating with a local Node.js backend ²⁶ ¹⁰ .

5. Stack Overflow – Tip on using “Extensions Reloader” for auto-reloading Chrome extensions during development ²⁰ .
6. Tomodachi Pets Docs – Development notes on using Chrome DevTools for extension logging and ensuring the local server is running ²² .

¹ ² ³ ¹⁹ Local LLM in the Browser Powered by Ollama | by Andrew Nguon | Medium

<https://medium.com/@andrewnguon/local-llm-in-the-browser-powered-by-ollama-236817f335da>

⁴ ⁵ ⁶ ⁷ Content scripts - Mozilla | MDN

https://developer.mozilla.org/en-US/docs/Mozilla/Add-ons/WebExtensions/Content_scripts

⁸ ¹⁰ ¹⁸ ²² ²³ ²⁴ ²⁵ ²⁶ GitHub - sumionochi/tomodachi-digital-pet-extension: Tomodachi Pets lets you draw or prompt custom pets/companions & their accessories, mint them as unique Pet NFT on the Sui network, personalize it with custom accessories generated by AI and bundle them onto your digital pets where they and selected assets will orbit around your cursor on any website! — all with full on-chain composability.

<https://github.com/sumionochi/tomodachi-digital-pet-extension>

⁹ ¹¹ ¹² Introduction - Ollama

<https://docs.ollama.com/api>

¹³ Generate a response - Ollama

<https://docs.ollama.com/api/generate>

¹⁴ ¹⁵ ¹⁶ ¹⁷ Structured Outputs - Ollama

<https://docs.ollama.com/capabilities/structured-outputs>

²⁰ javascript - How do I auto-reload a Chrome extension I'm developing? - Stack Overflow

<https://stackoverflow.com/questions/2963260/how-do-i-auto-reload-a-chrome-extension-im-developing>

²¹ Hot reloading Extensions using Webpack | Frontend Weekly - Medium

<https://medium.com/front-end-weekly/hot-reloading-extensions-using-webpack-cdfa0e4d5a08>